Part A: Broken Banking Transaction UI

1. Issues that could break or degrade production

Will break at runtime

1. **transactionService was not injected.** There is no constructor that injected transactionService, and we can see that it is used on ngOnInit(). This will throw an error.

2. **Categories search will crash if the value is null or undefined.** There is no type check or value check before using the category search value, and this will crash if the value is null or undefined. You can add type or value checks like “if (value) {}” or if (value !== null && value !== undefined) {}”

3. **No error handling on the subscription.** If the API fails or something goes wrong in the subscription, the error goes unhandled and the user sees an empty list with no explanation.

State handling

4. **Two sources of truth for the transaction data.** filteredTransactions is a derived state from the transaction data, but is stored separately. If getTransactions() is called again for some reason (maybe polling), “filteredTransactions = data” wipes the active search while the search box still shows text, so the UI and the input will disagree.

5. **No typescript safety.** transactions = [] is inferred as never[] (it has no type) under strict mode, and event: any with event.target.value loses all type safety. This means that any value can be passed and this can affect data integrity.

Subscriptions

6. **No unsubscribe in the subscription.** For a one-shot HTTP call this is mostly harmless, but in production transaction feeds are often long-lived (polling, websockets, etc). The subscription then outlives the component, causing memory leaks and callbacks writing to a destroyed component. Personally, I hate writing code without unsubscribing as this can cause huge performance issues that are hard to trace and can be caused by components not even in use anymore.

Performance

7. **Filtering on every input change.** (input) fires each character with no debounce, so typing “groceries” runs 9 full filters plus 9 change detection cycles.
8. **Rendering the entire list.** No pagination or virtual scrolling. A customer with 10k transactions gets 10k DOM nodes.

Template issue

9. **Case-sensitive and trimming the search.** “food” doesn’t match “Food”, and a trailing space returns nothing.
10. **Raw amount rendering.** {{ t.amount }} shows 1000.5 instead of ₦1,000.50. In banking, use the currency pipe with the transaction’s currency, and ideally handle money as minor units (kobo or cents) to avoid floating-point errors.
11. **No loading, empty, or error states.**

2. Fixes

a. State handling: I will keep one source of truth (the raw transactions) and *derive* the filtered list from it plus the search term, instead of storing it separately. Then new data automatically respects the current filter.

b. Subscriptions: I will let the async pipe own the subscription so it’s torn down on destroy. If you must subscribe manually, use takeUntilDestroyed().

c. Performance: For performance, the fixes will be to debounce the search, use OnPush, use trackBy, and use virtual scrolling (or server-side pagination and search for very large histories).

d. Template: Add null-safe rendering, currency pipe, loading/empty/error states, and a proper label.

NOTE: I have rewritten the component in a file that handles all the issues mentioned above, with comments.

Part B: Multi Bank Custom UI Engine

The frontend is built as an engine, and each bank is configured in its own config file. There is one codebase and one build. At startup, the app identifies the bank, loads its config, and uses it to decide the theme, layouts, onboarding flow, validation rules and enabled features. Adding or changing a bank means changing its config, not the code.

1. One codebase supports all banks

The app identifies the bank from the domain or from anywhere else we are identifying it and loads that bank's config before anything is rendered or shown (tenant-config.service.ts, app.config.ts). Configs are versioned JSON served by the backend and validated before use, with safe defaults, so changes don't need to be redeployed. All banks share the same component library and differ only in how components are configured, arranged and displayed.

2. UI changes dynamically per bank

- **Theming:** Each of the bank colours, fonts (if applicable), logos etc are stored as tokens in its config and applied as CSS variables at startup (theme.service.ts).
- **Layouts:** Pages are described in a config as a list of named blocks. A component registry maps names to lazy-loaded components, and a dynamic page component renders them (component-registry.ts, dynamic-page.component.ts).
- **Onboarding flows:** Each bank's onboarding is defined in the config as steps with conditional transitions. A flow engine walks through it like a state machine (flow-engine.service.ts, onboarding.component.ts).
- **Validation rules:** Rules are declared in config and converted into Angular validators through a validator registry, and forms are built from them (validator-registry.ts, dynamic-form.service.ts). The backend enforces the same rules, since frontend validation is only for user experience.

3. Features can be enabled or disabled per bank

A FeatureFlagService reads each of the bank's feature list and is used at three levels: route guards block disabled features so their code is never downloaded (feature.guard.ts), a *featureFlag directive hides UI (feature-flag.directive.ts), and layout blocks can be tied to a feature so they only render when it's enabled.

Production considerations

Bank-specific needs are built as new registered components and enabled through config, so the codebase is never forked. Configs are versioned with rollback and validated in CI. Tests run against every bank's config. Config controls what is shown, so the permissions and limits are always enforced on the backend.

NOTE: All code for this part is in a single file, with each section labelled by its file path.